Лабораторийн ажил №5

CPU хуваарилалт — Бүлэг 5, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Тэмдэглэгээ: Эргэлтийн хугацаа (turnaround time, TAT) = дууссан хугацаа − ирсэн хугацаа, Хүлээлтийн хугацаа (waiting time, WT) = TAT − burst time.

Дасгал 5.1

Асуулт. CPU хуваарилалтын алгоритм нь хуваарилагдсан процессуудын гүйцэтгэх дарааллыг тодорхойлдог. Нэг процессор дээр n процессыг хуваарилах бол хэдэн өөр хуваарь (schedule) боломжтой вэ? n-ээр илэрхийлсэн томьёо гарга.

Хариулт.

n! (n факториал) өөр хуваарь боломжтой.

Эхний байрлалд n процессын аль нэгийг, хоёр дахь байрлалд үлдсэн $n-1$-ийн аль нэгийг гэх мэтээр сонгоно. Иймд нийт $n \cdot (n-1) \cdots 2 \cdot 1 = n!$ сэлгэмэл (permutation) гарна. Жишээ нь 5 процесст $5! = 120$ хуваарь боломжтой.

Дасгал 5.2

Асуулт. Preemptive (буцаан авах) ба nonpreemptive (буцаан авахгүй) хуваарилалтын ялгааг тайлбарла.

Хариулт.

- **Nonpreemptive** хуваарилалтад CPU авсан процесс түүнийг өөрөө чөлөөлөх хүртэл ашиглана. Процесс дуусах эсвэл I/O хүлээх (waiting төлөвт шилжих) үед л CPU чөлөөлөгдөнө. Хэрэгжүүлэхэд хялбар ч урт процесс богино процессуудыг удаан хүлээлгэж, хариу үйлдлийн хугацааг уртасгадаг.
- **Preemptive** хуваарилалтад үйлдлийн систем ажиллаж буй процессоос CPU-г хүчээр авч болно. Жишээ нь квант (time quantum) дуусах (timer тасалдал), эсвэл илүү өндөр эрэмбэтэй процесс ready болох үед ингэдэг. Интерактив болон бодит цагийн системд зайлшгүй хэрэгтэй. Харин хуваалцсан өгөгдөлд race condition үүсэх эрсдэлтэй тул синхрончлол шаарддаг.

Жишээ: FCFS нь nonpreemptive, RR нь preemptive. SJF болон priority хуваарилалт аль ч хувилбартай байж болно.

Дасгал 5.3

Асуулт. Дараах процессууд заасан хугацаанд ирж, заасан хугацаагаар ажиллана. Nonpreemptive хуваарилалт ашиглаж, шийдвэр гаргах тухайн мөчид байгаа мэдээлэлд л тулгуурла.

Процесс	Ирэх хугацаа	Burst time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

(a) FCFS алгоритмаар дундаж эргэлтийн хугацаа хэд вэ? (b) SJF алгоритмаар хэд вэ? (c) SJF гүйцэтгэлийг сайжруулах ёстой ч бид P_1-ийг 0 мөчид эхлүүлсэн, учир нь удахгүй хоёр богино процесс ирэхийг мэдээгүй. CPU-г эхний 1 нэгж хугацаанд сул (idle) байлгаад дараа нь SJF хэрэглэвэл дундаж эргэлтийн хугацаа хэд болох вэ? Энэ хугацаанд P_1, P_2 хүлээж байгаа тул тэдний хүлээх хугацаа өсч болохыг санаарай. Энэ алгоритмыг future-knowledge scheduling гэж нэрлэж болно.

Хариулт.

(a) FCFS



TAT: $P_1 = 8 - 0 = 8$, $P_2 = 12 - 0.4 = 11.6$, $P_3 = 13 - 1 = 12$.
Дундаж TAT $= (8 + 11.6 + 12)/3 = 31.6/3 \approx \mathbf{10.53}$.

(b) SJF

0 мөчид зөвхөн P_1 ирсэн тул түүнийг эхлүүлнэ. 8 мөчид P_2 (4), P_3 (1) хүлээж байгаа бөгөөд хамгийн богино нь P_3.



TAT: $P_1 = 8$, $P_3 = 9 - 1 = 8$, $P_2 = 13 - 0.4 = 12.6$.
Дундаж TAT $= (8 + 8 + 12.6)/3 = 28.6/3 \approx \mathbf{9.53}$.

(c) Эхний 1 нэгж idle, дараа нь SJF



TAT: $P_3 = 2 - 1 = 1$, $P_2 = 6 - 0.4 = 5.6$, $P_1 = 14 - 0 = 14$.
Дундаж TAT $= (1 + 5.6 + 14)/3 = 20.6/3 \approx \mathbf{6.87}$.

CPU 1 нэгж сул байсан ч богино процессуудыг түрүүлж ажиллуулснаар дундаж эргэлтийн хугацаа 9.53-аас 6.87 болж буурсан. Гэхдээ ирээдүйн ирэлтийг мэдэх боломжгүй тул бодит системд үүнийг хэрэгжүүлэх боломжгүй.

Дасгал 5.4

Асуулт. Дараах процессуудын CPU burst time-ийг миллисекундээр өгөв. Процессууд P_1, P_2, P_3, P_4, P_5 дарааллаар бүгд 0 мөчид ирсэн гэж үзнэ.

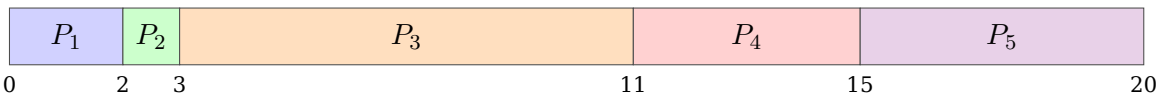
Процесс	Burst time	Priority
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

(a) FCFS, SJF, nonpreemptive priority (их тоо нь өндөр эрэмбэ) болон RR (квант $= 2$) алгоритмуудаар ажиллуулахыг харуулсан дөрвөн Gantt chart зур. (b) Алгоритм тус бүрт процесс бүрийн эргэлтийн хугацаа хэд вэ? (c) Процесс бүрийн хүлээлтийн хугацаа хэд вэ? (d) Аль алгоритм хамгийн бага дундаж хүлээлтийн хугацаа өгөх вэ?

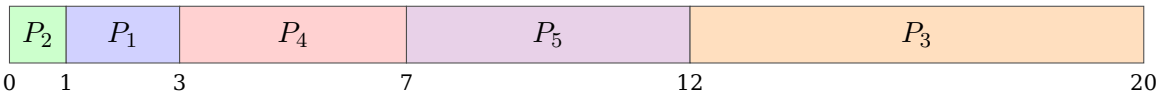
Хариулт.

(a) Gantt chart

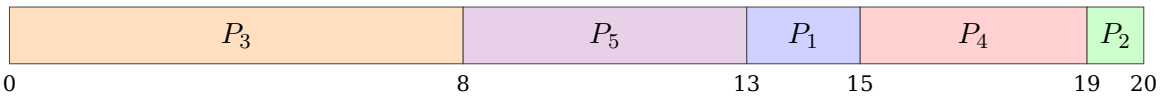
FCFS



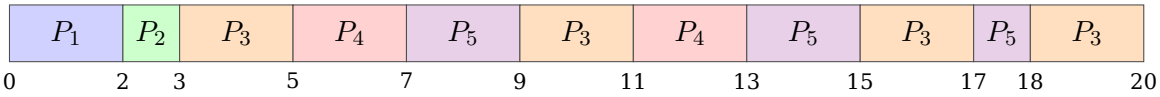
SJF



Nonpreemptive priority (P_1, P_4 хоёр ижил эрэмбэтэй тул FCFS дарааллаар)



RR (квант = 2)



(b) Эргэлтийн хугацаа (мс)

	P_1	P_2	P_3	P_4	P_5	Дундаж
FCFS	2	3	11	15	20	10.2
SJF	3	1	20	7	12	8.6
Priority	15	20	8	19	13	15.0
RR	2	3	20	13	18	11.2

(c) Хүлээлтийн хугацаа (мс)

	P_1	P_2	P_3	P_4	P_5	Дундаж
FCFS	0	2	3	11	15	6.2
SJF	1	0	12	3	7	**4.6**
Priority	13	19	0	15	8	11.0
RR	0	2	12	9	13	7.2

(d)

SJF алгоритм хамгийн бага дундаж хүлээлтийн хугацаа (4.6 мс) өгнө. SJF нь дундаж хүлээлтийн хугацааг хамгийн бага болгодог оновчтой (optimal) алгоритм гэдэг нь онолоор батлагдсан.

Дасгал 5.5

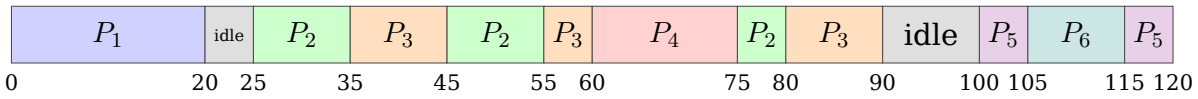
Асуулт. Дараах процессуудыг preemptive, round-robin хуваарилалтаар ажиллуулна.

Процесс	Priority	Burst	Ирэх хугацаа
P_1	40	20	0
P_2	30	25	25
P_3	30	25	30
P_4	35	15	60
P_5	5	10	100
P_6	10	10	105

Процесс бүр тоон эрэмбэтэй бөгөөд их тоо нь өндөр эрэмбийг илэрхийлнэ. Эдгээрээс гадна систем CPU нөөц зарцуулдаггүй idle task (P_{idle})-тай. Түүний эрэмбэ 0 бөгөөд өөр ажиллах процесс байхгүй үед хуваарилагдана. Квант (time quantum) 10 нэгж. Өндөр эрэмбэтэй процесс таслан зогсоовол зогсоогдсон процесс дарааллын төгсгөлд орно. (a) Gantt chart-аар хуваарилалтын дарааллыг харуул. (b) Процесс бүрийн эргэлтийн хугацаа хэд вэ? (c) Процесс бүрийн хүлээлтийн хугацаа хэд вэ? (d) CPU ашиглалтын хувь хэд вэ?

Хариулт.

(a) Gantt chart



Алхам алхмаар:

- 0–20: P_1 ганцаараа тул квант дуусахад ч үргэлжлүүлэн ажиллаж, 20-д дуусна. 20–25: ажиллах процесс байхгүй тул P_{idle} ажиллана.
- 25–35: P_2 ажиллана. 30-д ижил эрэмбэтэй P_3 ирж дараалалд орно. 35-д квант дуусахад P_2 (үлдэгдэл 15) дарааллын төгсгөлд орно.
- 35–45: P_3 (үлдэгдэл 15), 45–55: P_2 (үлдэгдэл 5), 55-аас P_3 ажиллана.
- 60: илүү өндөр эрэмбэтэй P_4 (35) ирж P_3-ийг таслан зогсооно. P_3-ийн үлдэгдэл 10 бөгөөд дарааллын төгсгөлд орно. 60–75: P_4 ажиллаж дуусна.
- 75–80: P_2 дуусна. 80–90: P_3 дуусна. 90–100: idle.
- 100–105: P_5 ажиллана. 105-д илүү өндөр эрэмбэтэй P_6 (10) ирж P_5-ийг таслан зогсооно. 105–115: P_6 дуусна. 115–120: P_5 дуусна.

(b), (c) Эргэлтийн ба хүлээлтийн хугацаа

Процесс	Ирсэн	Burst	Дууссан	TAT	WT
P_1	0	20	20	20	0
P_2	25	25	80	55	30
P_3	30	25	90	60	35
P_4	60	15	75	15	0
P_5	100	10	120	20	10
P_6	105	10	115	10	0
Дундаж				30	12.5

(d) CPU ашиглалт

Нийт 120 нэгж хугацаанаас CPU 20–25 болон 90–100 хооронд, нийт 15 нэгж сул байсан. Ажилласан хугацаа нь бүх CPU ээлжийн (burst) нийлбэр $20 + 25 + 25 + 15 + 10 + 10 = 105$.

$$\text{CPU ашиглалт} = \frac{105}{120} = \mathbf{87.5}\%.$$

Дасгал 5.6

Асуулт. Олон түвшинт дараалал (multilevel queue) бүхий системийн өөр өөр түвшинд өөр хэмжээтэй квант (time quantum) ашиглах нь ямар давуу талтай вэ?

Хариулт.

Процессын төрөл бүрт тохирсон квант өгч болдог нь гол давуу тал юм.

- **Өндөр эрэмбэтэй дараалал** (интерактив, I/O-bound процессууд) богино квант ашиглана. Эдгээр процесс CPU-г бага хугацаагаар хэрэглээд I/O руу шилждэг тул богино квант хариу өгөх хугацааг (response time) багасгана.
- **Бага эрэмбэтэй дараалал** (CPU-bound, batch процессууд) урт квант ашиглана. Эдгээр процесс CPU-г удаан ашигладаг тул урт квант нь орчин сэлгэлтийн (context switch) тоог, улмаар нэмэлт зардлыг (overhead) багасгаж, CPU-г үр ашигтай ашиглана.

Ингэснээр систем хариу үйлдлийн хурд болон үр ашгийг зэрэг хангаж чадна.

Дасгал 5.7

Асуулт. CPU хуваарилалтын олон алгоритм параметртэй байдаг. Жишээ нь RR алгоритмд time slice-ийг заах параметр хэрэгтэй. Multilevel feedback queue-д дараллын тоо, дараалал бүрийн хуваарилалтын алгоритм, процессыг дараалал хооронд шилжүүлэх шалгуур зэргийг тодорхойлох параметр хэрэгтэй. Тиймээс эдгээр нь үнэндээ алгоритмын олонлог юм (жишээ нь бүх time slice-тэй RR алгоритмуудын олонлог). Нэг олонлог нөгөөгөө агуулж болно: FCFS нь хязгааргүй time quantum-тэй RR алгоритм юм. Дараах хос алгоритмын олонлогуудын хооронд ямар хамаарал байх вэ (хэрэв байвал)? (a) Priority ба SJF, (b) Multilevel feedback queues ба FCFS, (c) Priority ба FCFS, (d) RR ба SJF

Хариулт.

(a) **Priority** $\supset$ **SJF.** SJF нь priority хуваарилалтын тусгай тохиолдол юм. Эрэмбийг дараагийн CPU ээлжийн (burst) таамагласан уртын урвуугаар тодорхойлдог: CPU ээлж богино байх тусам эрэмбэ өндөр.

(b) **MLFQ ⊃ FCFS.** FCFS нь multilevel feedback queue-ийн тусгай тохиолдол юм. Ганцхан дараалалтай, хязгааргүй квант бүхий, процесс шилжүүлэлтгүй MLFQ нь FCFS-тэй яг адил ажиллана. Мөн MLFQ-ийн хамгийн доод дараалал ихэвчлэн FCFS байдаг.

(c) **Priority ⊃ FCFS.** FCFS нь priority хуваарилалтын тусгай тохиолдол юм. Процесс эрт ирэх тусам эрэмбэ нь өндөр байхаар, эрэмбийг ирсэн хугацаагаар өгнө.

(d) **RR ба SJF: хамааралгүй.** Нэг нь нөгөөгөө агуулахгүй. RR нь CPU ээлжийн уртыг огт харгалздаггүй бол SJF нь зөвхөн CPU ээлжийн уртад тулгуурладаг. Аль ч параметртэй RR нь SJF-тэй ижил дараалал өгдөггүй, мөн эсрэгээрээ.

Дасгал 5.8

Асуулт. CPU хуваарилалтын алгоритм ойрын хугацаанд процессорыг хамгийн бага ашигласан процессуудыг илүүд үздэг гэж үзье. Энэ алгоритм яагаад I/O-bound програмуудыг илүүд үзэх боловч CPU-bound програмуудыг үүрд өлсгөхгүй (starvation) вэ?

Хариулт.

- **I/O-bound програмыг илүүд үзэх шалтгаан.** I/O-bound процесс CPU-г богино ээлжээр (burst) хэрэглээд I/O хүлээх төлөвт шилждэг. Тиймээс ойрын хугацааны CPU ашиглалт нь бага байж, алгоритм түүнд өндөр эрэмбэ өгнө.
- **CPU-bound програм өлсөхгүй шалтгаан.**
  1. I/O-bound процессууд ихэнх хугацааг I/O хүлээж өнгөрүүлдэг. Тэд блоклогдсон үед CPU сул болж, CPU-bound процесс ажиллах боломж олдоно.
  2. Алгоритм зөвхөн ойрын хугацааны CPU ашиглалтыг тооцдог. CPU-bound процесс CPU авалгүй удаан хүлээх тусам түүний ойрын ашиглалтын утга буурч, эрэмбэ нь аажмаар өснө. Эцэст нь тэр CPU-г заавал авна. Энэ нь aging механизмтай ижил үр нөлөөтэй.

Дасгал 5.9

Асуулт. PCS ба SCS хуваарилалтыг ялгаж тайлбарла.

Хариулт.

- **PCS (process-contention scope).** Урсгалын сан хэрэглэгчийн түвшний урсгалуудыг сул LWP-үүд дээр хуваарилна. Энд CPU-ийн төлөөх өрсөлдөөн нэг процессын урсгалуудын дунд явагдана. Many-to-one болон many-to-many загвартай системд хэрэглэгддэг бөгөөд ихэвчлэн программистын тогтоосон эрэмбээр хуваарилдаг.

- **SCS (system-contention scope, систем-өрсөлдөөний хүрээ).** Кернел кернелийн урсгалуудыг физик CPU дээр хуваарилна. Энд өрсөлдөөн систем дэх бүх урсгалын дунд явагдана. One-to-one загвартай системүүд (Linux, Windows) зөвхөн SCS ашигладаг.

Pthreads-д `pthread_attr_setscope()` функцээр `PTHREAD_SCOPE_PROCESS` (PCS) эсвэл `PTHREAD_SCOPE_SYSTEM` (SCS)-ийг сонгоно.

Дасгал 5.10

Асуулт. Уламжлалт UNIX хуваарилагч (scheduler) эрэмбийн тоо болон эрэмбэ хоёрын хооронд урвуу хамаарал тогтоодог: тоо их байх тусам эрэмбэ бага. Хуваарилагч секунд тутам процессуудын эрэмбийг дараах функцээр дахин тооцдог:

$$\text{Priority} = (\text{recent CPU usage}/2) + \text{base},$$

энд base $= 60$. Recent CPU usage нь эрэмбийг өмнө нь тооцсоноос хойш процесс CPU-г хэр олон ашигласныг илэрхийлнэ. P_1-ийн recent CPU usage 40, P_2-ийнх 18, P_3-ийнх 10 гэж үзье. Эрэмбийг дахин тооцоход эдгээр гурван процессын шинэ эрэмбэ хэд болох вэ? Эндээс уламжлалт UNIX хуваарилагч CPU-bound процессын харьцангуй эрэмбийг өсгөдөг үү, бууруулдаг уу?

Хариулт.

Процесс	Recent CPU usage	Тооцоо	Шинэ priority
P_1	40	$40/2 + 60$	**80**
P_2	18	$18/2 + 60$	**69**
P_3	10	$10/2 + 60$	**65**

Тоо их байх тусам эрэмбэ бага тул одоо P_3 хамгийн өндөр, P_1 хамгийн бага эрэмбэтэй болно.

Иймд уламжлалт UNIX хуваарилагч **CPU-bound процессын харьцангуй эрэмбийг бууруулдаг**. CPU-г их ашигласан процессын priority тоо өсч, эрэмбэ нь буурдаг. Харин CPU-г бага ашигладаг I/O-bound процессууд өндөр эрэмбэтэй болно.